Deep Learning-Based Deflickering of the Aurora Borealis: Using 3D CNNs to Remove U-Net-Induced Flicker in the AllSky Camera Dataset

Jason Press[1][0009-0000-3572-6336] and F.D.S Press[2]

[1] Keck Data Science Institute, Pepperdine University, Malibu, CA 90265
[2] Press Family, Belvedere, CA 94920

1 Abstract

The U-Net is a powerful architecture for performing deep learning-based image to image tasks. In a previous work, we extended the U-Net to the task of declouding videos of the aurora borealis. However,

2 Introduction

U-Nets are a very powerful tool for image segmentation and filtering. In a previous work, we utilized U-Nets to remove clouds from videos of the aurora borealis from the All-Sky Camera (ASC) dataset [12]. We achieved this by decomposing a video of the aurora borealis into individual frames and filtering each frame through a trained U-Net. To synthesize training data, we generate synthetic clouds using Perlin noise and overlay the synthetic clouds on an image with a clear sky to create cloudy and clear training pairs. The model is able to effectively extract auroral features from cloudy days [11].

However, the model has no concept of temporal consistency; it treats every frame as an individual image, not as a sequence of images over time. When an image I_t gets filtered, the model does not know of the relationship between I_{t-1}, I_t, and I_{t+1}, nor does the model know of the existence of I_{t-1} or I_{t+1}. This creates an image with no temporal consistency, also known as "flicker."

A number of solutions exist to reduce flicker in images. One solution is to use a U-Net that takes in I_{t-1}, I_t, and I_{t+1} rather than just a single image [1]. Incorporating temporal information into a U-Net has been used in tasks such as video pose estimation [9]. However, this approach would require completely retraining our previously established models.

Another solution is to use an external flickering reducing agent [8]. The pipeline would filter a video through the U-Net and then deflicker the filtered video. However, general deflickering agents are not designed to tackle the unique circumstances of U-Net-induced flicker, and greater efficiency and accuracy can be obtained with a dedicated, custom-made deflickering method.

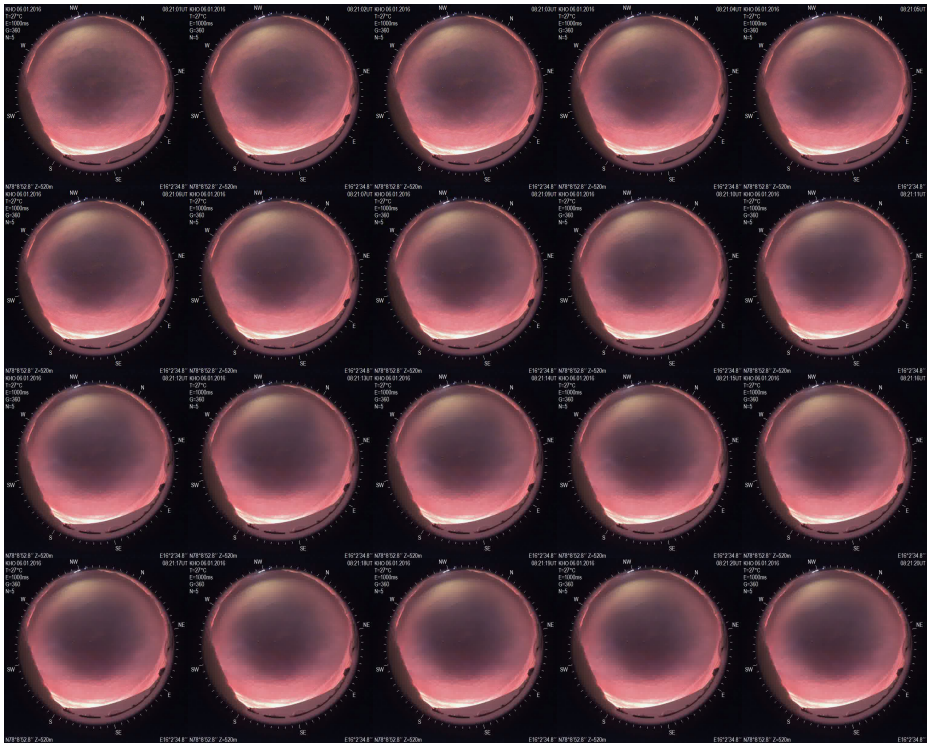

Fig. 1: A sequence of 20 images filtered through the U-Net. The images feature no temporal consistency. The flicker around the edge of the cleaned up clouds is different in each image.

A modular approach utilizes the U-Net to decloud the videos and a second model to reduce flickering in the output video. Two state-of-the-art solutions are the 3D Convolutional Neural Network (3D CNN) and the Long Short-Term Memory (LSTM) network. Both of these networks specialize in different applications [10]. The 3D CNN fits our use case best because of its focus on local discrepancies between individual frames rather than long-term flicker and pulsations.

The modular approach to creating a filtering pipeline has been previously implemented [4, 2]. This approach allows different deflickering techniques to be activated.

To train our network to reduce flicker, we utilize an unsupervised approach, as creating input-target data pairs is impossible. We utilize optical flow to calculate the amount of flicker between I_t and I_{t+1} [3, 13]. To help judge the quality of image-to-image loss, we employ Perceptual loss for its superior image-to-image loss metrics compared to pure L1, MSE, and other traditional distance metric-based loss functions [7]. We previously employed Perceptual Loss to act as our loss function in training the U-Net [6, 11].

This paper extends the 3D CNN to deflicker videos that have been filtered through a U-Net for noise removal and extends Optical Flow to train such a network.

3 Methods

To generate training data, we filter a large number of videos from the ASC dataset through the U-Net to produce videos that have been declouded. Doing so introduces the flicker that we will train the 3D CNN to eliminate. We then train the 3D CNN to minimize flicker.

The 3D CNN is given frames from $t - i$ to $t + i$ for some time t. We trained models with windows of $i = 3$, $i = 5$, and $i = 7$.

To calculate flicker, we utilize Optical Flow to warp the pixels from I_t to I_{t+1}. The temporal loss, or the amount of flicker, is the warp from I_t to I_{t+1} minus the actual I_{t+1}. The reconstruction loss is the difference between the predicted smooth I_t and the original I_t. Different loss metrics can be used for both temporal and reconstruction loss, such as L1 or Perceptual loss. We then define the total loss as

$$L_{\text{total}} = \lambda_t \cdot L_{L1\_temp} + \lambda_r \cdot L_{L1\_rec} + \lambda_p t \cdot L_{per\_temp} + \lambda_p r \cdot L_{per\_rec} \quad (1)$$

with a hyperparameter λ. The inclusion of reconstruction loss ensures that the model does not convert the images into pure black frames, as temporal loss only measures the amount of warping that needs to occur between the two frames. Reconstruction loss penalizes such an edge case. A larger λ increases the amount of flicker present but also results in less loss of the original details.

We use a combination of L1 and Perceptual loss because of their different tradeoffs. L1 loss excels in color accuracy but sacrifices sharpness. Perceptual loss excels in sharpness and overall reconstruction but introduces the same shimmering present in the U-Net videos. Tuning the lambdas yields the best of both loss functions without inducing their downsides.

We crop the metadata out of the frame to ensure the model does not filter the metadata, as the metadata must remain untouched. Including the metadata would unnecessarily penalize the model.

This can be easily achieved by multiplying the original image by a mask, where the mask is 0 where the metadata is and 1 where the video data is. Since all of our videos have the video data in the same center circle, we can statically define our mask for the multiplication, with I as the original image and M as the static mask.

$$I\prime = I \cdot M \quad (2)$$

When we put the video back together, we can add the original metadata back into the video, where M^{-1} is the inverse mask of M, and f_θ is the model:

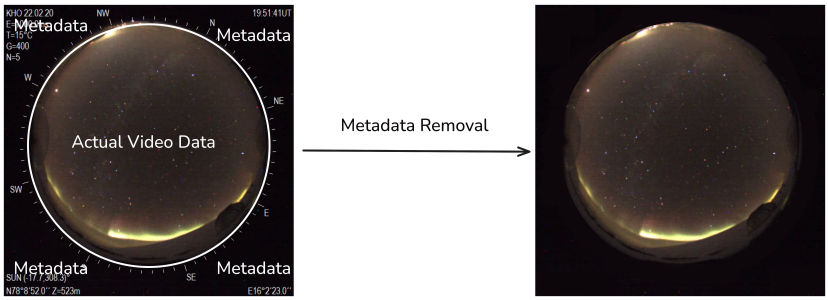

Fig. 2: Removing the metadata from an image from the ASC dataset. The metadata around the borders of the image gets masked over with the background color.

$$\hat{I} = f_\theta\left(I\prime\right) + I \cdot M^{-1} \tag{3}$$

This allows the model to affect only the video data without changing the metadata. Trying to learn on the metadata would be a waste of compute. The reason for this is left as an exercise for the reader.

3.1 Model Architecture

Our model follows a simple architecture, comprising of a head, body, and tail. The head converts the input features into a size the body can understand, with a 2D Convolution followed by ReLU. The body is a series of residual blocks, comprising of a 2D convolution, batch normalization, ReLU, another 2D convolution, and a final batch normalization. This output is combined with the original input, acting as a skip connection. The body consists of multiple residual blocks feeding one after another, along with a skip connection between each block. The tail takes the output of the body and converts it into a single frame for output, combined with the original frame at t, which acts like another skip connection.

Hyperparameters The model has a number of tunable hyperparameters: the number of residual blocks in the body; the hidden channels, or the number of filters in each Conv2d; the number of input frames to consider surrounding t (e.g. $i = 5 \implies$ input frames $= \{t-2, t-1, t, t+1, t+2\}$). We statically defined each kernel in the Conv2d to be 3, with a padding of 1. For training, we chose 12 residual blocks and 128 hidden channels. These are standard numbers used in 3D CNNs for larger images (512x512).

Additionally, the loss function has four different tunable λ values to weigh the L1 and Perceptual losses in both temporal and reconstruction. For training, we used $\lambda_t = 5.0$, $\lambda_r = 1.0$, $\lambda_{pt} = 0.5$, and $\lambda_{pr} = 1.0$.

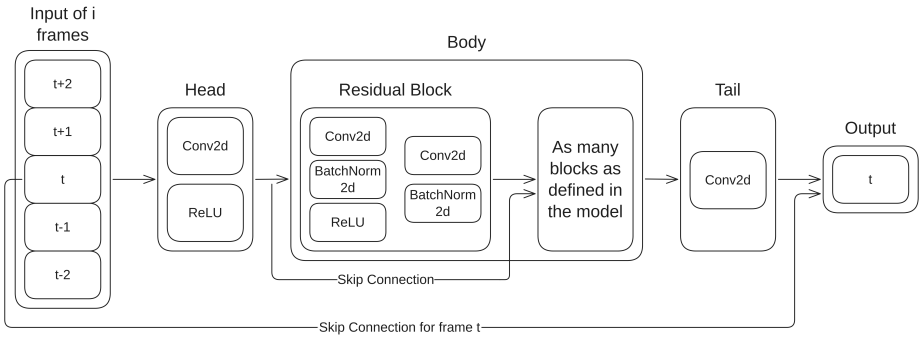

Fig. 3: Diagram of the model architecture. The input frames i get fed into a head, which feeds into a body comprised of residual blocks and skip connections, before being output to a tail with another skip connection.

3.2 Training data

To generate training data, we filtered 220 videos from various conditions through the U-Net, including various tunes of the U-Net, but most importantly with different α values for the synthetic cloud "diffusion" (between 0.0 and 1.0) and a random number of iterations (between 1 and 8). For training, we loaded a random subset of the videos (a total of 30 videos actively being trained on) into the GPU cluster and trained the model unsupervised on them. We trained with an 80% training and 20% validation split on the videos. No window from a video in the training set had a window from the same video in the validation set; the videos are exclusively in the training or validation set. For testing, we have a set of filtered videos that we did not provide to the model for training or validation.

4 Results

The objective of this model is to remove flicker induced by the U-Net. The temporal loss is the amount of flicker between frame t and $t + 1$. The goal of the 3D CNN is to minimize temporal loss while maintaining the original data. A successful model reduces flicker to a minimum while maintaining the original image. Because there is no ground truth, we cannot justify our results purely numerically.

We tested two different 3D CNNs: one with a 3 frame window and one with a 5 frame window. Both models have the same hyperparameters otherwise. Both models were trained for one epoch due to time constraints.

We compared the performance of these model to each other, to a baseline of no deflickering (i.e. just the U-Net), and the original video. A successful model should remove the constant U-Net flicker while leaving the aurora untouched. We include the original video to demonstrate a "baseline" for how much inherent flicker is in the scene.

4.1 Numerical Results

We used the same loss weight to calculate the flicker loss in testing as in training: $\lambda_t = 5.0$ and $\lambda_{pt} = 1.0$. Reconstruction is omitted because the reconstruction loss is always zero when comparing an image to itself. Figure 1 demonstrates the numerical loss of the original video in Figure 1, when it is filtered through the U-Net (as depicted in Figure 1), and then when it is smoothed by the 3D CNNs (the 3 frame window 3D CNN is depicted in Figure 4).

Video	**Total Loss**	**Frame 0**	**Frame 1**	**Frame 2**
No processing	0.5321	0.1338	0.1558	0.1944
U-Net	0.5663	0.2388	0.2520	0.2905
3D CNN (3 frame)	0.5848	**0.2231**	**0.2523**	**0.2829**
3D CNN (5 frame)	0.6192	0.2380	0.2537	0.2917

Table 1: Results of calculating loss over the video in Figure 1 in its original form, through just the U-Net, and then through the U-Net and 3D CNN with a 3 frame window. The raw numbers for the 3D CNN is better for the first frames, but is worse over the whole video compared to without the 3D CNN.

The 3D CNN does not perform as well as the U-Net in minimizing numerical flicker over the entire video. However, the first frames feature less numerical flicker in the 3D CNN video compared to the U-Net without smoothing. The video is not as numerically smooth as the original video, but the original video has clouds obscuring the aurora.

Moreover, the three frame window model outperforms the five frame window model. This is due to two factors: the smaller window model has less context, and can better focus on removing the flicker; and the small amount of training time both models had. With more training, the loss values for both 3D CNN models would be lower.

4.2 Visual Results

The videos smoothed by the model appear to be significantly smoother than the original videos. Although there is still some flickering present, the immediate harsh frame-to-frame flicker is nullified by the CNN, with the visual artifacts mostly being longer fluctuations over many frames. Figure 4 demonstrates the same twenty frames smoothed by the 3D CNN.

The five frame window model suffered from color shifting. This is due to the model's lack of training time. With more time, this issue would resolve itself.

5 Discussion

This model has a number of implications. Most importantly, it makes working with videos filtered through the U-Net in our previous work easier to observe.

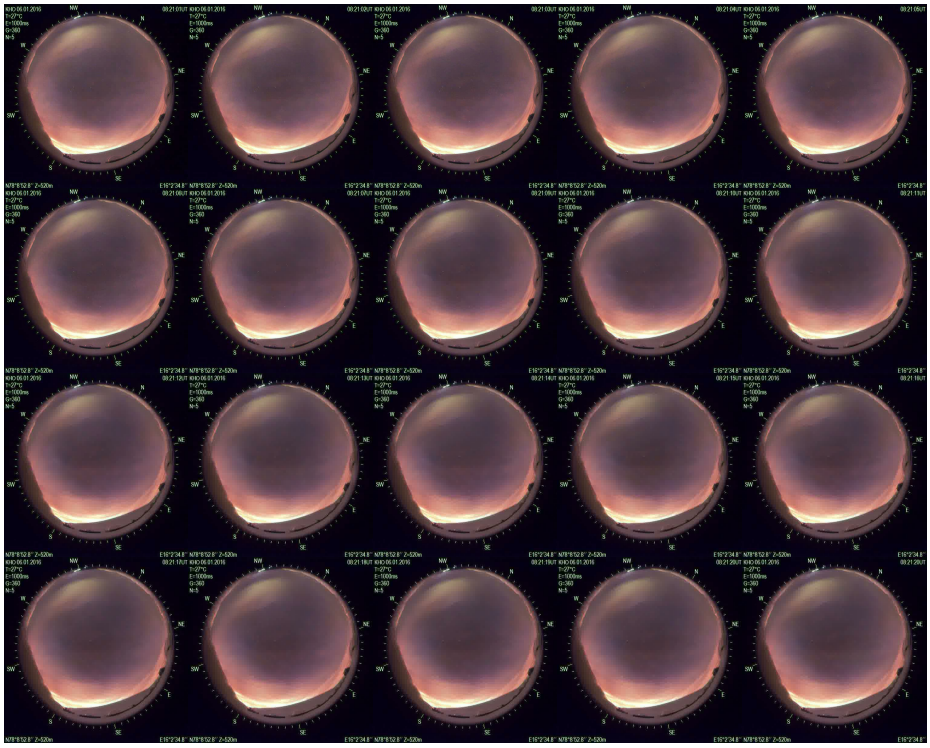

Fig. 4: Figure 1 smoothed by a three window figure. The images appear significantly smoother, with the immediate harsh flickering removed compared to Figure 1.

The largest element of human feedback we received as part of our previous work without temporal consistency was "it flickers a lot, and the flicker is distracting, and it gives me a headache when looking at the video repeatedly for long periods of time." This model directly addresses that flaw.

There are other possible approaches. The main flaw is the modularity of the system: why use multiple models when it can be combined into one? The reason behind this architectural choice is to allow for modularity. Sometimes, it is best for the scientists studying the aurora to analyze one frame with no context of the surrounding frames. Having modularity in the model architecture and the choice of what model to use for temporal smoothing allows for the personalization of choice. Some people may prefer no smoothing, and others may want smoothing. Having modularity allows for the extra flexibility.

Additionally, there are more problems with changing the original U-Net architecture. The synthetic cloud generation relies on static images, and does not account for the smooth movement and changing of clouds over time. While it is possible to change the synthetic cloud generation model to account for how clouds behave over time, it may yield worse results when using the DDPM-style noise introduction.

More importantly, the U-Net *does* already work: it effectively removes the clouds from videos. There is an old adage that says "if it ain't broke, don't fix it." So why fix what already works?

One big fear expressed by the scientists for whom this model is intended is the model hallucinating data. Hallucinations are a major drawback of blindly trusting Large Language Models, which are infamous for their hallucinations. This fear is the rationale for having a large reconstruction λ.

All of the 3D CNNs suffered from a lack of training time. With more training time, the model's performance would improve in all categories.

There is still more work that needs to be done. First, this paper only examines the results of one specific set of hyperparameters. Second, this paper only examines one architecture. More experimentation with hyperparameters, such as a larger window, could yield better results. Additionally, switching architectures to an LSTM or 3D U-Net could also work. The other option of changing the original U-Net and changing the synthetic noise generation to include how clouds behave dynamically could also be effective.

6 Conclusion

This paper introduces using 3D CNNs to remove U-Net induced flicker in videos of the aurora borealis. The model effectively learned how to remove the flicker from the videos introduced by the U-Net using a self supervised learning strategy. This model removes artifacts introduced by the cloud removing U-Net. There is still more room for improvement, such as tuning hyperparameters, but it introduces the final piece to create *Boredealis, a Complete System for Removing Clouds from the Aurora Borealis.*

Acknowledgments We would like to thank the W. M. Keck Foundation for their support to Pepperdine University and its Keck Data Science Institute.

References

1. Cicek, O., Abdulkadir, A., Lienkamp, S., Brox, T., Ronneberger, O.: 3d u-net: Learning dense volumetric segmentation from sparse annotation (06 2016). https://doi.org/10.48550/arXiv.1606.06650
2. Ebel, P., Garnot, V.S.F., Schmitt, M., Wegner, J.D., Zhu, X.X.: Uncrtaints: Uncertainty quantification for cloud removal in optical satellite time series (2023), https://arxiv.org/abs/2304.05464
3. Fischer, P., Dosovitskiy, A., Ilg, E., Häusser, P., Hazırbaş, C., Golkov, V., van der Smagt, P., Cremers, D., Brox, T.: Flownet: Learning optical flow with convolutional networks (2015), https://arxiv.org/abs/1504.06852
4. Haiyan, P., Chen, B., Zhou, R.: Physics-based noise modeling and deep learning for denoising permanently shadowed lunar images. Applied Sciences **15**, 2358 (02 2025). https://doi.org/10.3390/app15052358
5. Hetherington, J.H., Willard, F.D.C.: Two-, three-, and four-atom exchange effects in bcc ^{3}He. Phys. Rev. Lett. **35**, 1442–1444 (Nov 1975). https://doi.org/10.1103/PhysRevLett.35.1442, https://link.aps.org/doi/10.1103/PhysRevLett.35.1442
6. Iglovikov, V., Shvets, A.: Ternausnet: U-net with vgg11 encoder pre-trained on imagenet for image segmentation (2018), https://arxiv.org/abs/1801.05746
7. Johnson, J., Alahi, A., Fei-Fei, L.: Perceptual losses for real-time style transfer and super-resolution (2016), https://arxiv.org/abs/1603.08155
8. Lei, C., Ren, X., Zhang, Z., Chen, Q.: Blind video deflickering by neural filtering with a flawed atlas (2023), https://arxiv.org/abs/2303.08120
9. Li, W., Du, R., Chen, S.: Skeleton-based spatio-temporal u-network for 3d human pose estimation in video. Sensors **22**(7) (2022). https://doi.org/10.3390/s22072573, https://www.mdpi.com/1424-8220/22/7/2573
10. Mänttäri, J., Broomé, S., Folkesson, J., Kjellström, H.: Interpreting video features: a comparison of 3d convolutional networks and convolutional lstm networks (2020), https://arxiv.org/abs/2002.00367
11. Press, J., Scalzo, F., Fasel, G.: Deep learning-based declouding of the aurora borealis. Lecture Notes in Computer Science (2026)
12. Sigernes, F., Ivanov, Y., Chernouss, S., Trondsen, T., Roldugin, A., Fedorenko, Y., Kozelov, B., Kirillov, A., Safargaleev, V., Margit, D., Lorentzen, D., Oksavik, K.: A new auroral hyperspectral all-sky camera. Optical Society of America (2011), https://keoscientific.com/wp-content/uploads/Sigernes_et_al_NORUSCAII_Optics_express.pdf
13. Teed, Z., Deng, J.: Raft: Recurrent all-pairs field transforms for optical flow (2020), https://arxiv.org/abs/2003.12039

Other notes Jason Press would also like to thank his cat Sunny for his tremendous help with this paper. Jason did not want to change all of the "we" tenses to "I" tenses when writing this paper. The inclusion of a cat as a co-author in a paper for this reason has precedent in the literature [5].

Jason here. This is not part of my paper. This paper is part of a larger project, the one Dr. Fasel wants to submit to *Nature*. This is the final piece in the architecture puzzle.

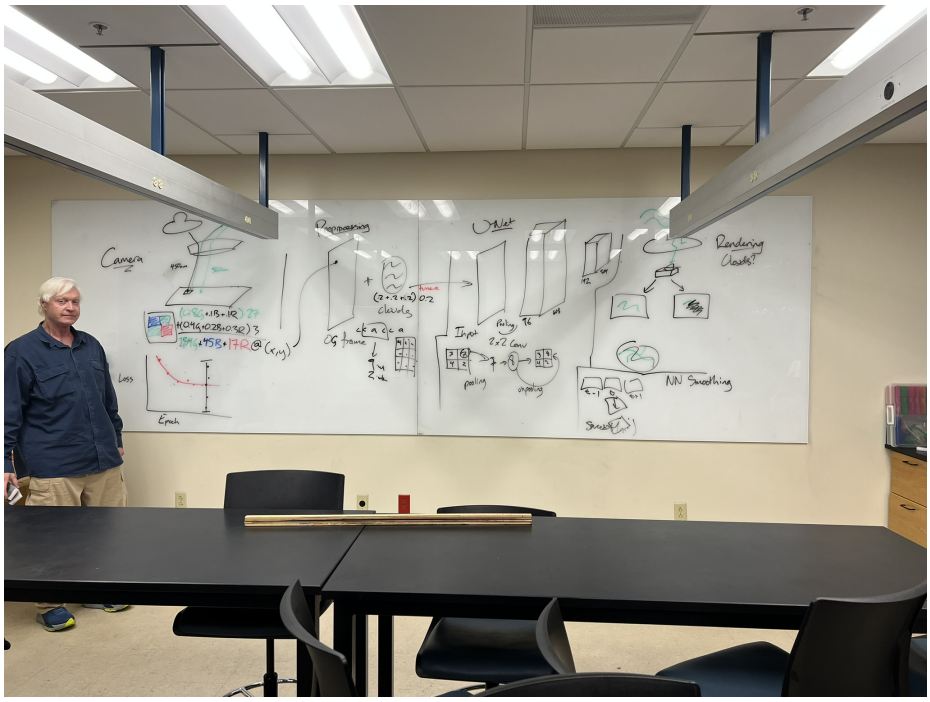

Fig. 5: The pipeline from camera, preprocessing, through the U-Net, ending with smoothing in the bottom right. The "Rendering Clouds?", Loss v. Epoch graph, and the crash course in how a camera works and U-Net filters/pooling were byproducts from this theorizing session. The model discussed in this paper is the bottom right model. The handsome gentleman on the left hand side needs no introduction.

That's why I used "we," spent time making a proper diagram for the 3D CNN, etc. We want to write the paper over break and submit it before the beginning of the next academic year, if possible.

A lot of this work, like the diagram, is going to be reused in my stage presentation at AGU25 as well as the *Nature* paper first draft, unless I spend my time making all of my diagrams look ultra-professional. But I don't particularly

want to learn the intricacies of Inkscape instead of Excalidraw (beyond using Inkscape to convert my Excalidraw SVGs to EPS files, but that's two clicks).

The last thing that needs to be done before I think this is ready for submission is the models need more training time. That's why Tailscale is so helpful. It will allow me to keep on doing work, even as I go home for Christmas break. Hopefully the Rule of Pi doesn't bite us in the ass.

Your feedback for this paper is extremely valuable to me. It will help the *Nature* draft. Please give me the feedback.

And *Boredealis* is a working title for my whole project, but it's been my working title since I started this project. Plus, names like "Uncrtaints" [2] are boring acronyms, and I don't want to do that.